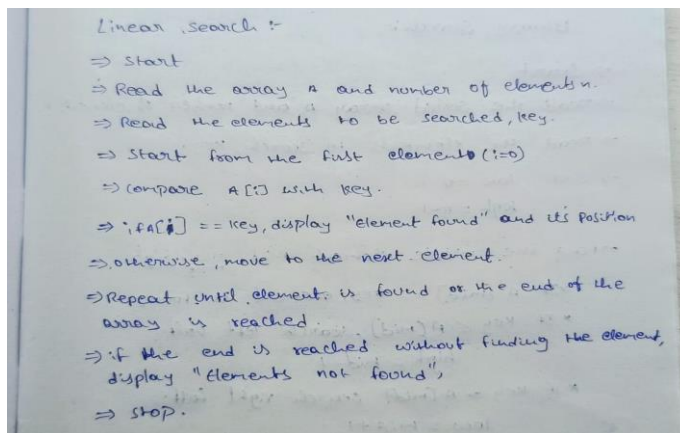


PRACTICAL-02

AIM: To implementation and time analysis of linear and binary search algorithm.

1.LINEAR SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main() {
    int arr[100], i, n, key;
    cout<<"enter number of array elements:"<<endl; cin>>n;
    cout<<"enter array elements:"<<endl;
    for(i=0; i<n; i++){
        cin>>arr[i];
    }

    cout<<"enter element to search:"<<endl;
    cin>>key;
    for(i=0; i<n; i++){
```

```
if(arr[i]==key)
{
    cout<<"element found at index of:"<<i;
}

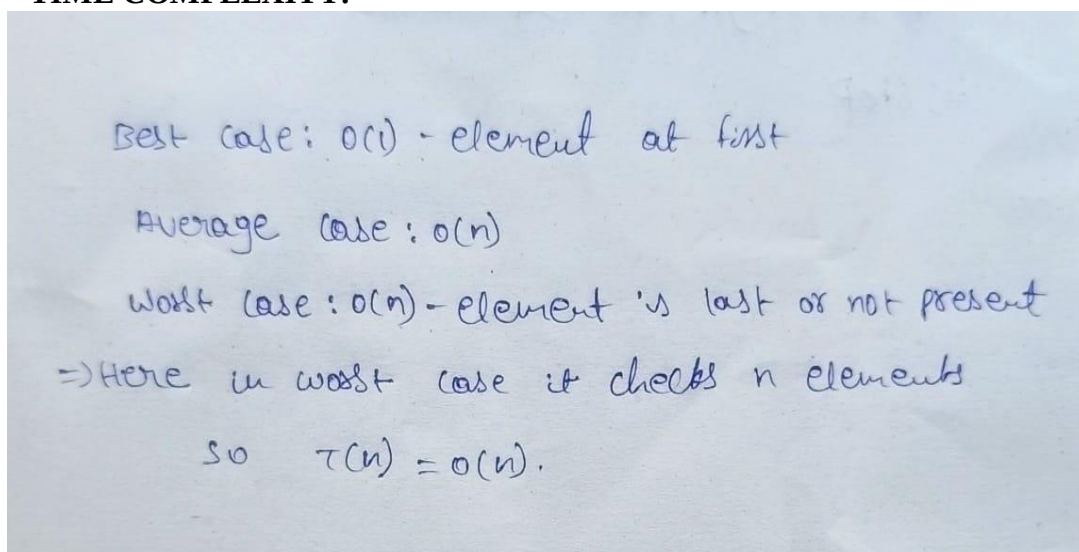
}

return 0;
}
```

OUTPUT:

```
enter number of array elements:
5
enter array elements:
7
9
5
34
78
enter element to serach:
5
element found at index of:2
```

TIME COMPLEXITY:



Best case: $O(1)$ - element at first
Average case: $O(n)$
Worst case: $O(n)$ - element is last or not present
 $\Rightarrow$ Here in worst case it checks n elements
So $T(n) = O(n)$.

2. BINARY SEARCH

ALGORITHM:

Binary Search:-

- ⇒ Start
- ⇒ Read the sorted array A and number of elements n.
- ⇒ Read the elements to search, key.
- ⇒ Set low = 0
high = n - 1
- ⇒ Find the middle element $mid = \frac{low + high}{2}$
 - * If $A[mid] == key$, element is found.
 - * If $key < A[mid]$, search left half:
 $high = mid - 1$
 - * If $key > A[mid]$ search right half:
 $low = mid + 1$
- ⇒ Repeat until $low > high$
- ⇒ If $low > high$, element is not found.
- ⇒ Stop.

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void bubblesort(int arr[], int n) {  
    for(int i=0; i<n-1; i++) {  
        for(int j=0; j<n-i-1; j++) {  
            if(arr[j] > arr[j+1]) {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;
```

```
    }  
    }  
}  
  
int binarySearch(int arr[], int n, int key) { int  
    low = 0, high = n-1;  
    while(low <= high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key) return mid;  
        else if(arr[mid] < key) low = mid+1;  
        else high = mid-1;  
    }  
    return -1;  
}  
  
int main() {  
    int arr[100], n, key;  
    cout << "Enter number of array elements: "; cin  
    >> n;  
  
    cout << "Enter array elements: ";  
    for(int i=0; i<n; i++) {  
        cin >> arr[i];  
    }  
  
    bubblesort(arr, n);  
  
    cout << "Sorted elements: ";
```

```
for(int i=0; i<n; i++) {  
    cout << arr[i] << " ";  
}  
  
cout << endl;  
cout << "Enter element to search: "; cin  
>> key;  
  
int result = binarySearch(arr, n, key);  
if(result != -1)  
    cout << "Element found at index " << result << endl;  
else  
    cout << "Element not found" << endl;  
  
return 0;  
}
```

OUTPUT:

```
Enter number of array elements: 6  
Enter array elements: 8  
4  
6  
1  
9  
19  
Sorted elements: 1 4 6 8 9 19  
Enter element to search: 6  
Element found at index 2
```

TIME COMPLEXITY:

Binary search:-

⇒ Binary searches only one half $\rightarrow T(n/2)$

⇒ the middle element comparison takes constant time $\rightarrow +1$

$$T(n) = T(n/2) + 1 \rightarrow \textcircled{1}$$

$n \rightarrow n/2$ Put in $\textcircled{1}$

$$T(n/2) = T(n/4) + 1 \rightarrow \textcircled{2}$$

$\textcircled{2}$ in $\textcircled{1}$

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/4) + 2 \rightarrow \textcircled{3}$$

$n \rightarrow n/4$ Put in $\textcircled{3}$

$$T(n/4) = T(n/8) + 1$$

$$T(n) = T(n/8) + 3$$

$$T(n) = T(n/2^k) + k$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

Apply log both sides

$$\log n = \log 2^k$$

$$k = \log n$$

$$T(n) = T\left(\frac{n}{2^{\log n}}\right) + \log n$$

$$= T\left(\frac{n}{2^{\log_2 n}}\right) + \log n$$

$$= T(1) + \log n$$

$$T(n) = O(\log n)$$

CONCLUSION:

Linear Search is simple and works on unsorted arrays but is slow with $O(n)$ time. Binary Search is much faster with $O(\log n)$ but requires the array to be sorted. For sorting, Bubble and Selection are basic $O(n^2)$, Insertion improves to $O(n)$ in best case, Quick Sort is efficient on average, and Merge Sort is consistently $O(n \log n)$ though it needs extra space.